CSS shape-outside Property: Complete Guide

Master Text Wrapping Around Custom Shapes

Contents

1. Introduction
   - (a) What is shape-outside? 2
   - (b) shape-outside vs. clip-path 2
2. Syntax and Basics 3
   - (a) Function Syntax 3
     - i. Basic Structure 3
     - ii. Examples 3
   - (b) Property Values 4
     - i. Available Functions 4
     - ii. Examples 4
   - (c) Float Requirement 5
     - i. Why Float is Needed 5
     - ii. Examples 5
3. Basic Shapes 6
   - (a) circle() - Circular Wrapping 6
     - i. Syntax and Usage 6
     - ii. Examples 6
   - (b) ellipse() - Elliptical Wrapping 7
     - i. Radii and Center 7
     - ii. Examples 7
   - (c) polygon() - Custom Shapes 8
     - i. Coordinate Definition 8
     - ii. Examples 9
   - (d) inset() - Box Model Wrapping 9
     - i. Rectangular Wrapping 9
     - ii. Examples 10
4. Advanced Techniques 10
   - (a) margin-box Reference 10

i. Box Model Integration 10
ii. Examples 11
(b) Image-Based Shapes 11
i. Using Images 11
ii. Examples 12
5. Real-World Use Cases 12
(a) Use Case 1: Circular Image with Text Wrap 12
(b) Use Case 2: Magazine-Style Layouts 13
(c) Use Case 3: Creative Article Designs 13
(d) Use Case 4: Product Showcases 14
6. Shape-Margin and Shape-Padding 14
(a) Adding Space Around Shapes 14
i. Shape-Margin 14
ii. Shape-Padding 15
(b) Examples 15
7. Browser Support & Compatibility 15
(a) Feature Support 15
(b) Vendor Prefixes 16
8. Best Practices 16
(a) When to Use shape-outside 16
(b) Performance Considerations 17
(c) Accessibility 17
9. Common Mistakes & Solutions 17
(a) Mistake 1: Missing Float 17
(b) Mistake 2: Incorrect Shape Values 18
(c) Mistake 3: Container Size Issues 18
10. Summary: Key Takeaways 18
11. Conclusion 19

1 Introduction

The CSS `shape-outside` property defines a shape that text content should wrap around. It allows content to flow around floated elements in non-rectangular patterns, enabling creative and sophisticated layouts.

1.1 What is shape-outside?

`shape-outside` determines the wrapping area for inline content around a floated element. Instead of rectangular wrapping, content can flow around circles, ellipses, polygons, and images.

Key characteristics:

- Works with floated elements only
- Defines wrapping boundary for text/inline content
- Supports basic shapes (circle, ellipse, polygon, inset)
- Can use image-based shapes for complex wrapping
- Separate from element's visual appearance
- Fully supported in modern browsers
- Enables magazine and editorial-style layouts

1.2 shape-outside vs. clip-path

Understanding the key differences:

Aspect	**shape-outside**	**clip-path**
Purpose	Text wrapping	Content clipping
Affects	Surrounding content	Element itself
Visibility	No visual change	Visible clipping
Requires float	Yes	No
Use case	Layout	Design effect
Content flow	Around shape	No flow

2 Syntax and Basics

2.1 Function Syntax

`shape-outside` uses straightforward function-based syntax.

2.1.1 Basic Structure

```
/* Basic syntax */
shape-outside: shape(parameters);

/* Examples */
shape-outside: circle(50%);
shape-outside: polygon(0 0, 100% 0, 100% 100%);
shape-outside: ellipse(40% 50%);
shape-outside: inset(10px);
```

Key rules:

- Element must have `float: left` or `float: right`
- Dimensions must be defined (width and height)
- Coordinates are relative to element
- Works with any floatable element

2.1.2 Examples

Example 1: Basic Circle Wrapping

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .circle-shape {
            width: 200px;
            height: 200px;
            background: #007BFF;
            float: left;
            shape-outside: circle(50%);
            margin-right: 20px;
        }

        p {
            column-count: 2;
        }
    </style>
</head>
<body>
    <div class="circle-shape"></div>
    <p>Text wraps around circular shape...</p>
</body>
</html>
```

2.2 Property Values

Available `shape-outside` functions and values.

2.2.1 Available Functions

```
/* Basic shapes */
shape-outside: circle(...);           /* Perfect circle */
shape-outside: ellipse(...);          /* Oval shape */
shape-outside: polygon(...);          /* Custom polygon */
shape-outside: inset(...);            /* Rectangle with inset */

/* Image-based shapes */
shape-outside: url('shape.png');      /* Image-based shape */
shape-outside: url('shape.svg');      /* SVG image */

/* Special values */
shape-outside: none;                  /* No shape (default) */
shape-outside: margin-box;            /* Use margin area */
```

2.2.2 Examples

Example 1: Different Shapes

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .shape {
            width: 150px;
            height: 150px;
            float: left;
            margin-right: 20px;
            margin-bottom: 20px;
            background: #007BFF;
        }

        .circle { shape-outside: circle(50%); }
        .ellipse { shape-outside: ellipse(50% 30%); }
        .polygon { shape-outside: polygon(50% 0%, 100% 50%, 50% 100%, 0% 50%);
            }
    </style>
</head>
<body>
    <div class="shape circle"></div>
    <p>Text around circle...</p>

    <div class="shape ellipse"></div>
    <p>Text around ellipse...</p>

    <div class="shape polygon"></div>
    <p>Text around polygon...</p>
</body>
</html>
```

2.3 Float Requirement

`shape-outside` only works with floated elements.

2.3.1 Why Float is Needed

```
/* Won't work: No float */
.no-float {
    width: 200px;
    height: 200px;
    shape-outside: circle(50%);  /* Ignored without float */
}

/* Works: With float */
.with-float {
    width: 200px;
    height: 200px;
    float: left;
    shape-outside: circle(50%);  /* Takes effect */
}
```

2.3.2 Examples

Example 1: Float Left and Right

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .float-left {
            width: 150px;
            height: 150px;
            float: left;
            background: #007BFF;
            shape-outside: circle(50%);
            margin-right: 20px;
        }

        .float-right {
            width: 150px;
            height: 150px;
            float: right;
            background: #FF6B6B;
            shape-outside: circle(50%);
            margin-left: 20px;
        }
    </style>
</head>
<body>
    <div class="float-left"></div>
    <p>Text wraps on the right...</p>

    <div class="float-right"></div>
```

```
    <p>Text wraps on the left...</p>
</body>
</html>
```

3 Basic Shapes

3.1 circle() - Circular Wrapping

Text wraps around circular shapes.

3.1.1 Syntax and Usage

```
/* circle(radius) - centered by default */
shape-outside: circle(50%);
shape-outside: circle(100px);
shape-outside: circle(50% at 50% 50%);  /* Explicit center */

/* Positioned circles */
shape-outside: circle(30% at 25% 25%);  /* Top-left */
shape-outside: circle(40% at 80% 80%);  /* Bottom-right */
```

3.1.2 Examples

Example 1: Centered Circle

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .circle-wrap {
            width: 200px;
            height: 200px;
            background: #007BFF;
            float: left;
            shape-outside: circle(50%);
            margin-right: 20px;
        }
    </style>
</head>
<body>
    <div class="circle-wrap"></div>
    <p>Text wraps smoothly around the circular element,
       creating an elegant and sophisticated layout that draws
       attention to both the shape and the content flowing around it.</p>
</body>
</html>
```

Example 2: Positioned Circle

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .circle-top-left {
            width: 200px;
            height: 200px;
            background: #FF6B6B;
            float: left;
```

```
            shape-outside: circle(40% at 30% 30%);
            margin-right: 20px;
        }
    </style>
</head>
<body>
    <div class="circle-top-left"></div>
    <p>Text wraps around a positioned circle...</p>
</body>
</html>
```

3.2 ellipse() - Elliptical Wrapping

Text wraps around oval-shaped areas.

3.2.1 Radii and Center

```
/* ellipse(rx ry) - centered by default */
shape-outside: ellipse(50% 30%);
shape-outside: ellipse(100px 60px);
shape-outside: ellipse(50% 30% at 50% 50%);  /* Explicit center */

/* Positioned ellipses */
shape-outside: ellipse(40% 50% at 30% 30%);  /* Top-left */
shape-outside: ellipse(60% 40% at 70% 60%);  /* Bottom-right */
```

3.2.2 Examples

Example 1: Wide Ellipse

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .ellipse-wide {
            width: 300px;
            height: 150px;
            background: #007BFF;
            float: left;
            shape-outside: ellipse(50% 50%);
            margin-right: 20px;
        }
    </style>
</head>
<body>
    <div class="ellipse-wide"></div>
    <p>Text wraps around a wide elliptical shape,
       allowing for creative editorial layouts...</p>
</body>
</html>
```

Example 2: Tall Ellipse

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .ellipse-tall {
            width: 150px;
            height: 300px;
            background: #FF6B6B;
            float: left;
            shape-outside: ellipse(30% 50%);
            margin-right: 20px;
        }
    </style>
</head>
<body>
    <div class="ellipse-tall"></div>
    <p>Text wraps around a tall elliptical shape...</p>
</body>
</html>
```

3.3 polygon() - Custom Shapes

Create custom multi-sided wrapping areas.

3.3.1 Coordinate Definition

```
/* polygon(x1 y1, x2 y2, x3 y3, ...) */
shape-outside: polygon(50% 0%, 100% 50%, 50% 100%, 0% 50%);  /* Diamond */
shape-outside: polygon(50% 0%, 100% 38%, 82% 100%, 18% 100%, 0% 38%);  /* Star
    */
shape-outside: polygon(0% 0%, 100% 0%, 100% 100%, 0% 100%);  /* Rectangle */

/* Triangle */
shape-outside: polygon(50% 0%, 100% 100%, 0% 100%);
```

3.3.2 Examples

Example 1: Diamond Shape

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .diamond {
            width: 200px;
            height: 200px;
            background: #007BFF;
            float: left;
            shape-outside: polygon(50% 0%, 100% 50%, 50% 100%, 0% 50%);
```

```
            margin-right: 20px;
        }
    </style>
</head>
<body>
    <div class="diamond"></div>
    <p>Text wraps around a diamond-shaped polygon...</p>
</body>
</html>
```

Example 2: Triangle Shape

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .triangle {
            width: 200px;
            height: 200px;
            background: #FF6B6B;
            float: left;
            shape-outside: polygon(50% 0%, 100% 100%, 0% 100%);
            margin-right: 20px;
        }
    </style>
</head>
<body>
    <div class="triangle"></div>
    <p>Text wraps around a triangular polygon...</p>
</body>
</html>
```

3.4 inset() - Box Model Wrapping

Define wrapping areas using box model offsets.

3.4.1 Rectangular Wrapping

```
/* inset(top right bottom left) - like margin/padding */
shape-outside: inset(20px);                  /* All sides */
shape-outside: inset(20px 40px);             /* Vertical Horizontal */
shape-outside: inset(20px 40px 30px 50px);   /* Top Right Bottom Left */

/* With rounded corners */
shape-outside: inset(20px round 10px);
```

3.4.2 Examples

Example 1: Simple Inset

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .inset-shape {
            width: 200px;
            height: 200px;
            background: #007BFF;
            float: left;
            shape-outside: inset(20px);
            margin-right: 20px;
        }
    </style>
</head>
<body>
    <div class="inset-shape"></div>
    <p>Text wraps with inset margins...</p>
</body>
</html>
```

4 Advanced Techniques

4.1 margin-box Reference

Control which box model area to reference.

4.1.1 Box Model Integration

```
/* Different reference boxes */
shape-outside: circle(50%) border-box;     /* Relative to border */
shape-outside: ellipse(50% 30%) padding-box;  /* Relative to padding */
shape-outside: polygon(...) content-box;   /* Relative to content */
shape-outside: circle(50%) margin-box;     /* Relative to margin */
```

4.1.2 Examples

Example 1: Different Box Models

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .box {
            width: 150px;
            height: 150px;
            float: left;
            margin: 20px;
            padding: 20px;
            border: 5px solid #333;
            background: #007BFF;
            margin-right: 20px;
        }

        .border-box {
            shape-outside: circle(50%) border-box;
        }

        .padding-box {
            shape-outside: circle(50%) padding-box;
        }

        .margin-box {
            shape-outside: circle(50%) margin-box;
        }
    </style>
</head>
<body>
    <div class="box border-box"></div>
    <p>Text with border-box reference...</p>

    <div class="box padding-box"></div>
    <p>Text with padding-box reference...</p>
```

```
    <div class="box margin-box"></div>
    <p>Text with margin-box reference...</p>
</body>
</html>
```

4.2 Image-Based Shapes

Use images to define complex wrapping boundaries.

4.2.1 Using Images

```
/* PNG image with transparency */
shape-outside: url('shape.png');

/* SVG image */
shape-outside: url('shape.svg');

/* With shape-image-threshold for transparency detection */
shape-outside: url('image.png');
shape-image-threshold: 0.5;  /* Pixels > 50% opacity define shape */
```

4.2.2 Examples

Example 1: Image-Based Wrapping

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .image-shape {
            width: 200px;
            height: 200px;
            float: left;
            background: url('photo.jpg') center/cover;
            shape-outside: url('photo.jpg');
            shape-image-threshold: 0.5;
            margin-right: 20px;
        }
    </style>
</head>
<body>
    <div class="image-shape"></div>
    <p>Text wraps around image contours...</p>
</body>
</html>
```

5 Real-World Use Cases

5.1 Use Case 1: Circular Image with Text Wrap

Create elegant layouts with circular images.

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .profile {
            width: 200px;
            height: 200px;
            float: left;
            background: url('profile.jpg') center/cover;
            border-radius: 50%;
            shape-outside: circle(50%);
            margin-right: 30px;
            margin-bottom: 30px;
        }

        .bio {
            text-align: justify;
            line-height: 1.6;
        }
    </style>
</head>
<body>
    <div class="profile"></div>
    <div class="bio">
        <h2>Author Bio</h2>
        <p>Text flows naturally around the circular profile image,
           creating a professional and polished magazine-style layout.
           The circular shape draws attention to the image while allowing
           the text to flow seamlessly around it.</p>
    </div>
</body>
</html>
```

5.2 Use Case 2: Magazine-Style Layouts

Create sophisticated editorial designs.

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .article-image {
            width: 250px;
            height: 300px;
            float: left;
            background: url('article.jpg') center/cover;
            shape-outside: polygon(0 0, 100% 0, 80% 100%, 0 100%);
            margin-right: 30px;
```

```
        }

        .article {
            column-count: 2;
            column-gap: 30px;
        }
    </style>
</head>
<body>
    <div class="article">
        <div class="article-image"></div>
        <p>Article content with custom shaped wrapping...</p>
    </div>
</body>
</html>
```

5.3 Use Case 3: Creative Article Designs

Unique shape wrapping for visual interest.

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .decorative-shape {
            width: 200px;
            height: 200px;
            float: left;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            shape-outside: circle(30% at 30% 30%);
            margin-right: 20px;
        }
    </style>
</head>
<body>
    <div class="decorative-shape"></div>
    <p>Creative wrapping with positioned circles...</p>
</body>
</html>
```

5.4 Use Case 4: Product Showcases

Wrap text around product images.

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .product-image {
            width: 250px;
            height: 250px;
            float: left;
            background: url('product.png') center/contain no-repeat;
```

```
            shape-outside: url('product.png');
            shape-margin: 10px;
            margin-right: 20px;
        }
    </style>
</head>
<body>
    <div class="product-image"></div>
    <p>Product description text wraps around the product image...</p>
</body>
</html>
```

6 Shape-Margin and Shape-Padding

6.1 Adding Space Around Shapes

Control spacing between shape boundary and text.

6.1.1 Shape-Margin

```
/* Add space outside the shape */
shape-outside: circle(50%);
shape-margin: 20px;        /* Adds 20px gap */

/* Multiple values */
shape-margin: 10px 20px 15px 25px;
```

6.1.2 Examples

Example 1: Margin Around Circle

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .circle-margin {
            width: 150px;
            height: 150px;
            float: left;
            background: #007BFF;
            shape-outside: circle(50%);
            shape-margin: 20px;
            margin-right: 20px;
        }
    </style>
</head>
<body>
    <div class="circle-margin"></div>
    <p>Text maintains distance from the circular shape...</p>
</body>
```

```
</html>
```

Example 2: Asymmetric Shape-Margin

```
<!DOCTYPE html>
<html>
<head>
    <style>
        .asymmetric-margin {
            width: 150px;
            height: 150px;
            float: left;
            background: #FF6B6B;
            shape-outside: circle(50%);
            shape-margin: 10px 30px 20px 15px;
            margin-right: 20px;
        }
    </style>
</head>
<body>
    <div class="asymmetric-margin"></div>
    <p>Text with asymmetric spacing around shape...</p>
</body>
</html>
```

7 Browser Support & Compatibility

7.1 Feature Support

Feature	Chrome	Firefox	Safari	Edge	Opera
Basic shapes	Yes	Yes	Yes	Yes	Yes
Image shapes	Yes	Limited	Yes	Yes	Yes

7.2 Vendor Prefixes

Use `-webkit-` prefix for broader compatibility.

```
-webkit-shape-outside: circle(50%);
shape-outside: circle(50%);
```

8 Best Practices

8.1 When to Use shape-outside

- Magazine and editorial layouts
- Creative product showcases

- Circular or artistic image presentations
- Content-focused designs where shape adds value
- When visual impact is important

8.2 When NOT to Use

- Simple rectangular layouts (too complex)
- Accessibility-critical content (text flow may be confusing)
- Mobile-first designs (consider responsive complexity)
- When regular float/grid is sufficient

8.3 Performance Considerations

- Minimal performance impact
- Simple shapes faster than complex polygons
- Image-based shapes require image loading
- No significant rendering cost
- Well-supported GPU acceleration

8.4 Accessibility

- `shape-outside` doesn't affect DOM order
- Screen readers read content in DOM order, not visual order
- Ensure logical content order in markup
- Test with screen readers for comprehension
- Provide clear text flow and hierarchy

9 Common Mistakes & Solutions

9.1 Mistake 1: Missing Float

Problem: `shape-outside` has no effect without float.

```
/* Wrong: No float specified */
.element {
    width: 150px;
    height: 150px;
    shape-outside: circle(50%);  /* Ignored! */
}
```

```
/* Correct: Float is required */
.element {
    width: 150px;
    height: 150px;
    float: left;
    shape-outside: circle(50%);  /* Works */
}
```

9.2 Mistake 2: Incorrect Shape Values

Problem: Shape values outside element dimensions.

```
/* Problematic: Circle larger than container */
.element {
    width: 100px;
    height: 100px;
    float: left;
    shape-outside: circle(100%);  /* Too large */
}

/* Better: Circle fits container */
.element {
    width: 100px;
    height: 100px;
    float: left;
    shape-outside: circle(50%);  /* Optimal size */
}
```

9.3 Mistake 3: Container Size Issues

Problem: Element dimensions not specified.

```
/* Wrong: No dimensions */
.element {
    float: left;
    background: blue;
    shape-outside: circle(50%);  /* Size undefined */
}

/* Correct: Dimensions specified */
.element {
    width: 150px;
    height: 150px;
    float: left;
    background: blue;
    shape-outside: circle(50%);  /* Size defined */
}
```

10 Summary: Key Takeaways

1. **Float is required:** `shape-outside` only works with floated elements

2. **Define dimensions:** Always set width and height on floated element
3. **Choose appropriate shape:** circle, ellipse, polygon, or inset
4. **Use shape-margin:** Add space between shape and text for readability
5. **Consider accessibility:** Ensure logical content order
6. **Test layouts:** Different browsers may render slightly differently
7. **Image-based shapes:** Great for complex, organic shapes
8. **Responsive design:** Plan for how layout adapts on mobile

11 Conclusion

The CSS `shape-outside` property enables sophisticated text wrapping around floated elements, opening up creative possibilities for web design. Whether creating magazine-style layouts with circular images or building editorial designs with custom polygons, `shape-outside` provides the control needed for modern, visually compelling websites.

By combining `shape-outside` with proper design principles and accessibility considerations, you can create layouts that are both beautiful and functional. Master this property and elevate your web design capabilities to professional editorial standards!